Operating Systems — Study Guide

Topics 4 onward: IPC, System Calls, Memory, Files, I/O, Security, Networking and Embedded Linux

This companion guide continues after the previous PDF on Process Management, CPU Scheduling, and Synchronization & Deadlocks. It follows the broad topic sequence from your 275-slide syllabus and highlights concepts that are useful for exam revision. The two uploaded E3 papers visibly emphasise IPC, system calls and memory fundamentals; several later syllabus areas were not meaningfully tested in those papers, so they are included here for syllabus coverage rather than because they appeared frequently in the papers.

4. Interprocess Communication (IPC) and Signals

IPC allows processes to exchange data and coordinate activity. Processes may have separate address spaces, so the OS provides mechanisms for communication.

4.1 Common IPC mechanisms

Mechanism	How it works	Typical use / caution
Pipe	Byte stream, commonly between related processes	Simple parent-child data flow; usually unidirectional
FIFO (named pipe)	Pipe with a filesystem name	Can connect processes that are not directly related
Shared memory	Processes map a common memory region	Fast for large data; requires synchronization
Message queue	Messages are queued and received by processes	Preserves message boundaries; queue limits apply
Signal	Asynchronous notification of an event	Good for notifications, not bulk data transfer
Socket	Communication endpoint, local or networked	Useful between processes on different machines too

4.2 Message passing

Message-passing systems commonly provide two fundamental operations: **send(message)** and **receive(message)**. Communication may be direct (naming the peer) or indirect (using a mailbox/queue), and may be blocking or non-blocking depending on the API.

4.3 Pipes example

```
int fd[2];
pipe(fd);                 // fd[0]: read end, fd[1]: write end
// Parent/child can close the unused end,
// then use read(fd[0], ...) and write(fd[1], ...).
```

Always check return values, close unused file descriptors, and handle end-of-file/errors. A pipe does not automatically solve synchronization for shared state.

4.4 Signals

• Signals notify a process that an event occurred; examples include SIGINT and SIGTERM.

• A signal handler should do only safe, minimal work; many library functions are not async-signal-safe.

• Use signals for notification/control, not as a substitute for a data channel.

Exam focus: compare pipes, shared memory, message queues and signals; know send/receive; understand why shared memory needs synchronization.

5. System Calls and Library Calls

A system call is a controlled interface through which a user program requests a service from the OS kernel. A library call is a function provided by a library; it may or may not invoke a system call.

5.1 Common Linux/POSIX calls

Purpose	Examples	Notes
Process creation/control	fork(), exec*(), wait(), waitpid(), _exit()	exec replaces the current process image; fork creates a child
File I/O	open(), read(), write(), close()	Use file descriptors; check return values
Memory	mmap(), munmap(), brk() (lower-level)	Allocators such as malloc() are library functions
Process information	getpid(), getppid()	Return process identifiers
I/O control	ioctl()	Device-specific control operations

5.2 System call vs library call

• A system call crosses into kernel-controlled service code; the exact mechanism is architecture and OS dependent.

• A library function executes in user space unless it needs a kernel service.

• Examples: read() is a system call interface; printf() is a C library function that formats output and may eventually use write().

• fopen()/fclose() are standard C library interfaces; they are not themselves the same as open()/close() system calls.

5.3 Return values and errors

Check return values. Many system calls return -1 on error and set errno; read() and write() can transfer fewer bytes than requested. Robust programs handle interrupted calls and partial I/O when appropriate.

```
ssize_t n = read(fd, buffer, sizeof buffer);
if (n == -1) {
    perror("read");
} else if (n == 0) {
    /* end of file */
} else {
    /* n bytes were read */
}
```

Exam focus: purpose of system calls, fork/exec/wait, file operations, library-call distinctions, and error handling.

6. Memory Management

Memory management tracks which memory is in use, allocates and releases memory, and provides processes with protected address spaces.

6.1 Logical vs physical address

• A logical/virtual address is generated by program execution and interpreted within a process's address space.

• A physical address refers to a location in main memory (RAM).

• The memory-management unit (MMU) translates addresses using structures such as page tables.

6.2 Paging

Paging divides virtual memory into fixed-size pages and physical memory into frames. Page tables map virtual page numbers to physical frame numbers. Paging avoids external fragmentation but can cause internal fragmentation, especially in the final page of an allocation.

6.3 Segmentation

Segmentation divides a program's address space into logical variable-sized segments such as code, data and stack. Segment-based schemes can reflect program structure but may suffer external fragmentation.

6.4 Swapping and allocation

• Swapping moves a process or memory pages between RAM and secondary storage, depending on the OS mechanism.

• Dynamic allocation requests memory while a program runs; allocation failures and lifetime management must be handled.

• Stack commonly holds call frames, return addresses, parameters and automatic/local variables; exact layout depends on ABI/compiler.

6.5 Page faults

A page fault occurs when a memory reference cannot be completed using the current page-table state. The OS determines whether the access is valid and, if needed, loads the page and updates mappings. A page fault is not necessarily an error; demand paging intentionally loads pages on first use.

Exam focus: logical vs physical addresses, paging, segmentation, swapping, demand paging, page faults and stack contents.

7. Virtual Memory

Virtual memory lets a process use an address space larger or more flexible than the RAM currently assigned to it. Only needed portions may be resident at a given time.

7.1 Demand paging

With demand paging, a page is loaded when first referenced. Benefits include less initial I/O and memory use for pages never touched. Costs include page-fault handling and storage I/O when a needed page is absent.

7.2 Page-replacement algorithms

Algorithm	Rule	Key idea
FIFO	Evict oldest loaded page	Simple; can exhibit Belady's anomaly
Optimal	Evict page used farthest in future	Benchmark ideal; future knowledge unavailable in practice
LRU	Evict least recently used page	Uses recent past as a proxy for near future
Clock/Second chance	Use reference bits in a circular scan	Practical approximation to LRU

7.3 Thrashing and TLB

• Thrashing occurs when excessive page faults cause the system to spend too much time moving pages rather than executing useful work.

• A Translation Lookaside Buffer (TLB) caches recent address translations to speed up virtual-to-physical translation.

• Working set describes the pages a process actively uses over a time window; insufficient frames for active working sets can increase faulting.

Exam focus: demand paging, page faults, replacement algorithms, TLB and thrashing.

8. File Systems

A file system organizes persistent data into files and directories and manages metadata, storage allocation, permissions and consistency.

8.1 File concepts and operations

• File metadata may include size, timestamps, ownership, permissions and storage references.

• Common operations include create, open, read, write, seek, close, rename and delete.

• Directories map names to file-system objects; pathnames identify objects within the directory hierarchy.

• File descriptors are small process-local integers returned by calls such as open().

8.2 Allocation approaches

• **Contiguous allocation:** consecutive blocks; fast sequential access but can cause external fragmentation and growth difficulty.

• **Linked allocation:** blocks link to the next block; flexible growth but poor random access.

• **Indexed allocation:** an index maps file offsets to data blocks; supports direct access with index overhead.

8.3 Permissions and reliability

Unix-like permissions commonly distinguish owner, group and others, with read/write/execute bits. File-system reliability techniques can include journaling, checksums, backups and clean shutdown/recovery mechanisms; exact features vary by file system.

8.4 Buffered I/O

Standard I/O functions such as fopen(), fread(), fwrite() and fclose() are library interfaces and often use buffering. Low-level calls such as open(), read(), write() and close() operate on file descriptors.

Exam focus: file operations, directory structure, allocation methods, permissions, file descriptors and library vs system I/O.

9. I/O Systems and Device Interaction

The OS provides a consistent interface to devices while handling differences in hardware, timing, errors and transfer methods.

9.1 Core concepts

• **Device driver:** software that controls a particular device or controller.

• **Interrupt:** hardware/software event that requests CPU attention.

• **Polling:** CPU repeatedly checks device status; simple but can waste CPU cycles.

• **DMA:** Direct Memory Access lets a capable device/controller transfer blocks to/from memory with limited CPU copying.

• **Buffering:** temporary storage used to handle speed or size differences.

• **Caching:** keeps copies of data to accelerate repeated access.

• **Spooling:** queues work for a device that processes jobs serially, such as a printer.

9.2 Typical I/O path

```
Application
    |
System call / library interface
    |
Kernel I/O subsystem
    |
Device driver
    |
Controller / hardware device
    |
Completion via interrupt, status check or DMA-related event
```

9.3 Embedded considerations

• Understand device registers, memory-mapped I/O and interrupt service routines where relevant to the platform.

• Keep interrupt handlers short; defer longer processing to a suitable thread/task or work queue.

• Handle timeouts, partial transfers, device removal/failure and power-state changes.

• In real-time systems, bound the time spent in critical paths and avoid unbounded blocking where deadlines matter.

Exam focus: interrupts vs polling, drivers, DMA, buffering/caching, and how an application request reaches hardware.

10. Security

Operating-system security aims to protect confidentiality, integrity and availability while allowing authorised use.

10.1 Core principles

• **Least privilege:** give each process/user only the permissions needed.

• **Authentication:** establish identity; **authorization:** determine allowed actions.

• **Isolation:** separate processes and privilege levels to limit damage.

• **Access control:** permissions, access-control lists and capabilities can control resource use.

• **Defense in depth:** use multiple complementary controls.

10.2 Common mechanisms

• User/kernel privilege separation and controlled system-call interfaces.

• File permissions and ownership; avoid running services with unnecessary privileges.

• Secure boot and signed software where supported by platform design.

• Memory protections such as address-space separation, non-executable memory and ASLR where available.

• Logging, patching, secrets handling, secure configuration and recovery planning.

10.3 Embedded-system security

Embedded devices should minimise enabled services, validate inputs, protect update mechanisms, authenticate software updates, restrict debug interfaces in production and define recovery behaviour for failed updates. Security measures must be designed alongside safety and availability requirements.

Exam focus: least privilege, authentication vs authorization, isolation, access control, secure boot and update security.

11. Networking and Distributed Systems

Networking lets systems exchange data; distributed systems coordinate components that may run on different machines and may fail independently.

11.1 Basic terms

• **IP address:** identifies a network interface/addressing endpoint at the IP layer.

• **Port:** identifies an application/service endpoint on a host.

• **TCP:** connection-oriented, reliable byte stream.

• **UDP:** datagram-oriented, no built-in delivery/order guarantee.

• **Client-server:** client requests a service; server listens and responds.

• **Latency:** time for data or a request to travel/complete; **throughput:** data processed/transferred per unit time.

11.2 Distributed-system concerns

• Network delay and disconnection; retries and timeouts.

• Partial failure: one component may fail while others continue.

• Duplicate requests and idempotency.

• Ordering, synchronization and consistency of shared data.

• Authentication, encryption and input validation across trust boundaries.

Exam focus: IP address, ports, TCP vs UDP, client-server roles and common distributed-system failure modes.

12. Socket Programming

A socket is a communication endpoint. In a common TCP server, the server creates a socket, binds it to a local address/port, listens, accepts a client, exchanges data, and closes descriptors.

12.1 Typical TCP server sequence

```
socket()   -> create endpoint
bind()     -> associate local IP/port
listen()   -> mark socket as passive/listening
accept()   -> accept a client connection
read()/recv(), write()/send() -> exchange bytes
close()    -> release descriptors
```

12.2 Typical TCP client sequence

```
socket()   -> create endpoint
connect()  -> connect to server IP/port
send()/write(), recv()/read() -> exchange bytes
close()    -> release descriptor
```

12.3 Important details

• The client must know how to reach the server, typically the server IP address/name and port.

• A TCP socket is a byte stream; one send() does not necessarily correspond to one recv().

• Check all return values, handle partial sends/receives, and define message framing.

• Use htons()/htonl() and related functions when converting multi-byte protocol values between host and network byte order.

• Do not assume a single read() receives the entire application message.

Exam focus: purpose of socket(), bind(), listen(), accept() and connect(); server addressing; TCP stream behaviour.

13. OS Fundamentals, Boot and Performance

13.1 OS purpose and types

An operating system manages hardware resources and provides abstractions/services for programs. Common categories include batch, time-sharing, distributed and real-time systems. Embedded OS design places particular emphasis on resource constraints, device interaction and often predictable timing.

13.2 Kernel, shell and boot

• **Kernel:** core privileged component managing CPU, memory, devices and system calls.

• **Shell:** user interface/interpreter for commands; it is not the kernel.

• **Boot flow:** reset/firmware (BIOS or UEFI) → bootloader → kernel → early user-space/init/service manager.

• On Linux, the first user-space process is traditionally called init; modern systems often use a service manager such as systemd as PID 1.

• ELF is a common executable/object-file format on Linux.

13.3 Performance metrics

Metric	Meaning
CPU utilisation	Fraction of time CPU is busy doing work
Throughput	Number of jobs completed per unit time
Turnaround time	Completion time minus arrival/submission time
Waiting time	Time spent waiting in ready queues (in scheduling exercises)
Response time	Time until the first response/first CPU service, according to context
Dispatch latency	Time to switch from one process to another

Desirable goals: generally maximise CPU utilisation and throughput while minimising turnaround, waiting and response times; real-time systems additionally care about meeting deadlines.

14. Embedded Linux Industry Readiness

Embedded Linux work includes bringing up the system, deploying applications, monitoring services, debugging faults and designing recovery behaviour.

14.1 Practical skills

• Read service and kernel logs; use tools such as journalctl, dmesg, ps, top, free, df and systemctl where available.

• Inspect process state and resource use; identify runaway CPU, memory exhaustion and blocked tasks.

• Use exit codes, logs, core dumps and a debugger to investigate crashes.

• Use watchdogs and supervisor/service restart policies carefully; prevent endless restart loops.

• Understand cross-compilation, target architecture, libraries, packaging and deployment constraints.

• Keep update/recovery mechanisms safe, testable and robust against power loss.

14.2 Debugging workflow

```
Observe symptom
    -> collect logs and reproduction steps
    -> check process/service status and resource use
    -> isolate recent changes and dependencies
    -> reproduce or minimise the failure
    -> fix and add a regression test
    -> verify recovery and document the result
```

14.3 Common scenarios to practise

• Application crashes after startup: inspect logs, exit status and core dump; check configuration and library dependencies.

• System hangs: identify blocked threads/processes, lock waits, I/O waits and resource exhaustion.

• Service repeatedly restarts: inspect failure reason, restart policy, dependencies and back-off behaviour.

• Device I/O fails: inspect driver/kernel messages, permissions, device node, timeouts and return codes.

Exam and interview focus: debugging method, log-based analysis, service management, crash recovery, watchdogs and deployment basics.

Practice MCQs — Topics 4 onward

1. Which IPC mechanism is especially suitable for high-volume data sharing when processes coordinate access?

A. Shared memory

B. Signal only

C. PID

D. CPU register

2. Which pair represents fundamental message-passing operations?

A. open/close

B. send/receive

C. fork/exit

D. bind/listen

3. Which statement about exec() is correct?

A. Always creates a new PID

B. Replaces the current process image on success

C. Waits for all children

D. Allocates a semaphore

4. Which is a C library function that commonly uses lower-level I/O internally?

A. printf()

B. fork()

C. waitpid()

D. execve()

5. Paging maps virtual pages to:

A. CPU cores

B. Physical frames

C. File names

D. Network ports

6. A page fault always means an illegal memory access.

A. True

B. False

7. Which replacement algorithm evicts the page that has not been used for the longest time in the past?

A. FIFO

B. LRU

C. Optimal

D. FCFS

8. Which file allocation approach uses an index to locate a file's data blocks?

A. Indexed allocation

B. Round Robin

C. FIFO scheduling

D. Spooling

9. DMA is used to:

A. Transfer data between device and memory with reduced CPU copying

B. Schedule processes

C. Encrypt files automatically

D. Resolve deadlock

10. The principle of least privilege means:

A. Give every user administrator rights

B. Grant only the permissions needed

C. Disable logging

D. Share all memory

11. TCP is best described as:

A. Reliable connection-oriented byte stream

B. Unordered shared memory

C. A file system

D. A process state

12. Which call typically marks a TCP server socket as listening?

A. connect()

B. listen()

C. read()

D. fork()

13. Which component loads/starts the kernel during boot?

A. Bootloader

B. PCB

C. Semaphore

D. Socket

14. Throughput measures:

A. Jobs completed per unit time

B. Time to first response only

C. Memory address width

D. Number of processes blocked

15. What should be checked first when an embedded Linux service repeatedly crashes?

A. Ignore logs

B. Inspect logs, exit status and configuration

C. Disable all security

D. Increase CPU priority without diagnosis

Answer key

1-A · 2-B · 3-B · 4-A · 5-B · 6-B · 7-B · 8-A · 9-A · 10-B · 11-A · 12-B · 13-A · 14-A · 15-B

Study note: These explanations extend beyond the exact wording of the two exam PDFs to cover the remaining syllabus areas. The original papers visibly support basic IPC, system-call, paging/swapping, boot, performance and socket topics; the other sections are included to help you prepare the broader syllabus, not because those papers establish their exam frequency.